УНИВЕРЗИТЕТ
У НОВОМ САДУ

ФАКУЛТЕТ
ТЕХНИЧКИХ НАУКА

Трг Доситеја Обрадовића 6, 21000 Нови Сад, Република Србија
Деканат: 021 6350-413; 021 450-810; Централа: 021 485 2000
Рачуноводство: 021 458-220; Студентска служба: 021 6350-763
Телефакс: 021 458-133; e-mail: ftndean@uns.ac.rs

ИНТЕГРИСАНИ
СИСТЕМ
МЕНАЏМЕНТА
СЕРТИФИКОВАН ОД:

Funkcionalna verifikacija hardvera

NAZIV PROJEKTA:

Funkcionalna verifikacija sistema za detekciju lica upotrebom Viola-Jones metodologije

PROJEKAT IZRADILI:

Radivoje Jović EE 202/2021

Vojislav Sandić

Sara Glišević

Sadržaj

1. Opis dizajna koji se verifikuje 4
1.1 Interfejsi dizajna 4
1.2 Registarska mapa 4
1.3 Organizacija memorije i adresna putanja 5
2. Verifikacioni plan 6
2.1 Reset i inicijalizacija 6
2.2 AXI4-Lite protokol 6
2.3 Registarski fajl 7
2.4 Semantika kontrolnih bitova 7
2.5 Adresna putanja row banaka 8
2.6 Funkcionalnost kaskade 8
3. Struktura verifikacionog okruženja 9
3.1 Uloga virtuelnog sekvencera 10
3.2 Komponente okruženja 10
3.3 Redoslijed kompajliranja paketa 10
4. Referentni model i zlatni vektori 12
4.1 Skupovi zlatnih vektora 13
4.2 Shadow model registarskog fajla 13
5. Testovi 14
5.1 Test rotacije kao invarijanta 14
6. Rezultati verifikacije 15
7. Analiza funkcionalne pokrivenosti 18
7.1 Pokrivenost po grupama 18
7.2 Nedostaci okruženja otkriveni mjerenjem pokrivenosti 19
7.3 Nazadovanje pokrivenosti izazvano boljim vektorima 19
7.4 Pokrivenost naspram stvarne provjere 20
8. Regresioni testovi 21
9. Pokretanje simulacije 21
9.1 Vivado simulator (xsim) 21

9.2 Xcelium simulator 22
10. Analiza grešaka 23
10.1 Greške pronađene u verifikacionom okruženju.....23
10.2 Hipoteze iz statičke analize.....23
10.3 Planirano injektiranje grešaka 24
11. Ograničenja verifikacije.....24
12. Zaključak.....25
Literatura.....26

1. Opis dizajna koji se verifikuje

Dizajn koji se verifikuje je hardverski sistem za detekciju lica baziran na Viola-Jones metodologiji, implementiran u VHDL jeziku i namijenjen izvršavanju na Xilinx Zybo Z7-10 platformi zasnovanoj na čipu xc7z010clg400-1.

Jezgro nad prozorom veličine 24x24 piksela izvršava kaskadu od 25 faza i ukupno 2913 slabih klasifikatora, i donosi odluku da li prozor sadrži lice. Kaskadna struktura je ključna za performanse, jer se velika većina prozora odbacuje već u prvoj fazi, nakon svega 9 klasifikatora, dok punu kaskadu prolaze samo prozori koji zaista liče na lice.

Jezgro ne radi direktno sa pikselima nego sa integralnom slikom. Softver unaprijed izračuna integralnu i kvadriranu integralnu sliku i u jezgro upisuje 25 redova integralne slike, što odgovara visini detekcionog prozora uvećanoj za jedan, i dva reda kvadrirane integralne slike koja se koriste za izračunavanje lokalne varijanse.

Osnovna motivacija za hardversku akceleraciju je količina posla, jer se za detekciju na cijeloj slici kroz trinaest nivoa piramide skaliranja evaluira preko 200.000 prozora, što softverska implementacija ne može da izvrši u realnom vremenu.

1.1 Interfejsi dizajna

Dizajn ima dva magistralna interfejsa sa jasno razdvojenim ulogama. Ta podjela je najvažnija činjenica za arhitekturu verifikacionog okruženja, jer zahtijeva dva nezavisna agenta čije se aktivnosti ne smiju preklapati.

Interfejs	Uloga	Zašto postoji odvojeno
S00_AXI	AXI4-Lite slave, kontrolni i statusni registri	Mali broj pristupa, ali je potrebna proizvoljna adresibilnost registara
BRAM_PORTA	Upload redova integralne slike	Jedini prenos visokog opterećenja u sistemu; AXI BRAM kontroler ga obavlja približno jednom riječju po taktu
irq	Level-high prekid, prati CTRL.DONE	Jedan red slike odgovara jednom prekidu u režimu sweep-a

Tabela 1. Magistralni interfejsi dizajna

1.2 Registarska mapa

Procesor upravlja jezgrom preko jedanaest registara mapiranih na AXI4-Lite interfejs. U sistemu su registri mapirani na baznu adresu 0x40000000, a AXI BRAM kontroler na 0x42000000, što je preuzeto iz hardverskog opisa sistema u datoteci vj_system_wrapper.xsa.

Bitno je uočiti da su pojedina polja u jezgru uža nego što je širina registra u wrapperu. ROW_BASE je petobitan, a X_START, X_LAST, X_STEP i HIT_ADDR devetobitni. Upis vrijednosti šire od polja mora biti odsječen, jer bi u suprotnom nevalidna vrijednost bila proslijeđena dalje u dizajn.

Ofset	Naziv	Tip	Opis
0x00	CTRL	RW	b0 RESET (self-clearing), b1 START (self-clearing), b2 MODE, b4 DONE (briše se upisom jedinice)
0x04	X_START	RW	x koordinata prozora u režimu A, prvi x u režimu B; 9 bita
0x08	WIDTH	RW	Širina tekuće skale; full 32-bitni registar
0x0C	RESULT	RO	Režim A: 1 = lice, −i = odbijen na fazi i
0x10	X_LAST	RW	Režim B: posljednji x koji se testira; 9 bita
0x14	X_STEP	RW	Režim B: korak pomjeranja; 9 bita, reset vrijednost 1
0x18	HIT_COUNT	RO	Režim B: broj detekcija u hit baferu
0x1C	HIT_DATA	RO	Režim B: x koordinata detekcije na poziciji HIT_ADDR
0x20	ROW_BASE	RW	Rotacija 25 row banaka; 5 bita, dozvoljeno 0-24
0x24	STATUS	RO	b0 BUSY
0x28	HIT_ADDR	RW	Indeks u hit baferu; 9 bita

Tabela 2. Registarska mapa AXI4-Lite interfejsa

1.3 Organizacija memorije i adresna putanja

Redovi integralne slike smješteni su u 27 nezavisnih BRAM banaka: 25 banaka drži redove integralne slike, a dvije banke sa fiksnim indeksima 25 i 26 drže logičke redove 0 i 23 kvadrirane integralne slike. Bajt adresa koju generiše AXI BRAM kontroler dekoduje se na sljedeći način:

```
adresna rijec = bram_addr_a(15 downto 2)
    biti 13..9  -> indeks banke (0..24 int, 25 = sq red 0, 26 = sq red 23)
    biti  8..0  -> kolona
```

Banka sa indeksom 26 zahtijeva bit 15 bajt adrese. Adresni dekoder uži od potrebnog ne bi mogao da adresira gornje banke, a simulacija bi i dalje naizgled radila, pa je provjera adresne putanje izdvojena kao zasebna stavka verifikacionog plana.

Registar ROW_BASE omogućava rotaciju banaka. Time se pri pomjeranju detekcionog prozora po y osi ne prepisuju sve banke, nego samo ona koja drži najstariji red, dok se ROW_BASE uvećava za jedan. Rotacija se u jezgru računa funkcijom rot_bank, jednim uslovnim oduzimanjem koje je ispravno samo ako su oba operanda manja od 25.

Jezgro podržava dva režima rada. U režimu A obrađuje se jedan prozor na poziciji X_START, a rezultat se čita iz registra RESULT. U režimu B jezgro samostalno pomjera x od X_START do X_LAST korakom X_STEP i bilježi detekcije u hit bafer dubine 512.

2. Verifikacioni plan

Verifikacioni plan definiše šta se provjerava, kojim mehanizmom i kojim testom. Organizovan je po funkcionalnostima dizajna, a ne po komponentama verifikacionog okruženja, jer je cilj pokriti ponašanje dizajna, a ne strukturu testbencha.

Za svaku stavku plana navedeno je da li se provjera vrši poređenjem sa referentnim modelom, poređenjem sa shadow modelom registarskog fajla, ili mjerenjem funkcionalne pokrivenosti.

Verifikacioni plan treba dati odgovor na neka osnovna pitanja kao što su:
Da li se registri pravilno inicijalizuju / resetuju ?
Da li sistem ispravno radi u terminima AXI protokola (ispravan handshaking, ispravne širine polja…) ?
Da li sistem ispravno reaguje na START signal kao impuls ?
Da li se DONE registar čisti upisom jedinice ?
Da li sistem ispravno funkcioniše i za nasumično generisane vrijednosti ulaznih stimulusa?
Da li sistem ispravno radi i pravi razliku između oba režima rada (detekcije jednog prozora i sweepa redova) ?
Kako se sistem ponaša u situacijama uzastopnog sweepovanja ?
Da li mehanizam rotacije banaka ispravno radi u svim realno ostvarivim scenarijima ?

2.1 Reset i inicijalizacija

Nakon hardverskog reseta svi registri sa mogućnošću upisa moraju imati definisane reset vrijednosti. Registar X_STEP je jedini čija reset vrijednost nije nula.

Stavka provjere	Mehanizam provjere	Test
Reset vrijednosti svih RW registara	Shadow model registarskog fajla	vj_test_reset
Read-only registri vraćaju nule	Poređenje sa očekivanom nulom	vj_test_reset
Reset posle popunjavanja registara	Shadow model	vj_test_reset
Soft reset (CTRL bit 0) vraća jezgro u idle	Čitanje STATUS.BUSY	vj_test_ctrl
Soft reset prekida obradu u toku	Čitanje STATUS.BUSY posle prekida	vj_test_ctrl

Tabela 3. Verifikacioni plan - reset i inicijalizacija

2.2 AXI4-Lite protokol

Slave u dizajnu podiže AWREADY tek kada su AWVALID i WVALID istovremeno visoki. Implementacija drivera koja bi čekala jedan pa drugi kanal zaglavila bi se, pa je redoslijed kanala eksplicitno pokriven.

Stavka provjere	Mehanizam provjere	Test

Handshake sa adresom prije podataka	Coverage grupa cg_axil_write, bin addr_first	vj_test_regs
Handshake sa podacima pre adrese	Bin data_first	vj_test_regs
Istovremena adresa i podaci	Bin together	svi testovi
BVALID se zadržava do BREADY	Bin cp_bready::delayed	vj_test_regs
RVALID i RDATA se zadržavaju do RREADY	Bin cp_rready::delayed	vj_test_regs
BRESP i RRESP su uvijek OKAY	Provjera u scoreboard-u i illegal_bins	svi testovi

Tabela 4. Verifikacioni plan - AXI4-Lite protokol

2.3 Registarski fajl

Registar koji je u wrapperu širi nego u jezgru propagirao bi nevalidnu vreijdnost dalje u dizajn, a običan read-back to ne bi otkrio. Zbog toga se širine polja provjeravaju eksplicitno.

Stavka provjere	Mehanizam provjere	Test
Upis i read-back svakog RW registra	Shadow model	vj_test_regs
ROW_BASE je 5-bitan (upis 33 vraća 1)	Shadow model sa maskom [4:0]	vj_test_regs
X_START, X_LAST, X_STEP su 9-bitni	Shadow model sa maskom [8:0]	vj_test_regs
HIT_ADDR je 9-bitan	Shadow model sa maskom [8:0]	vj_test_regs
Upis u read-only registre nema efekta	Shadow se ne mijenja, read-back potvrđuje	vj_test_regs
CTRL pri čitanju vraća samo MODE i DONE	Maska nad pročitanom vrijednošću	svi testovi

Tabela 5. Verifikacioni plan - registarski fajl

2.4 Semantika kontrolnih bitova

Ove provjere razdvajaju ispravnu implementaciju od tri uobičajene greške: START tretiran kao nivo umesto kao impuls, DONE koji se briše pri čitanju umjesto upisom jedinice, i DONE koji se gubi pri upisu u drugi registar.

Stavka provjere	Mehanizam provjere	Test
START je impuls, ne nivo	STATUS.BUSY mora biti 0 poslije mirovanja	vj_test_ctrl
DONE preživljava čitanje CTRL registra	Scoreboard prati stanje done_seen	vj_test_ctrl
DONE preživljava upis u drugi registar	Isto	vj_test_ctrl
DONE se briše isključivo upisom jedinice	Isto	vj_test_ctrl
DONE se ne pojavljuje bez prethodnog START-a	Scoreboard prati run_started	svi testovi
irq je aktivan kada je DONE postavljen	irq se sampluje uz svaku transakciju	svi testovi

Tabela 6. Verifikacioni plan - kontrolna logika

2.5 Adresna putanja row banaka

Port A svake banke dijele upisna putanja sa magistrale i col0 čitanje jezgra, uz pretpostavku da se nikada ne preklapaju. To je bitno ograničenje dizajna, pa ga scoreboard aktivno nadgleda.

Stavka provjere	Mehanizam provjere	Test
Riječ stiže u banku i kolonu koje diktira adresa	Monitor dekoduje banku i kolonu iz magistrale	svi testovi
Svih 25 integralnih banaka su adresirane	cg_bram_write, bins int_row[]	vj_test_mode_a
Obje kvadrirane banke (25 i 26) su adresirane	bins sq_row0, sq_row23	vj_test_mode_a
Adresiran je cijeli opseg kolona	cross x_bank_col	vj_test_ctrl, vj_test_full
Nema upisa u banke dok je jezgro zauzeto	Scoreboard prijavljuje grešku	svi testovi

Tabela 7. Verifikacioni plan - adresna putanja

2.6 Funkcionalnost kaskade

Ovo je jezgro verifikacije koje vrši poređenje izlaza dizajna sa nezavisnim referentnim modelom, prozor po prozor, u oba režima rada.

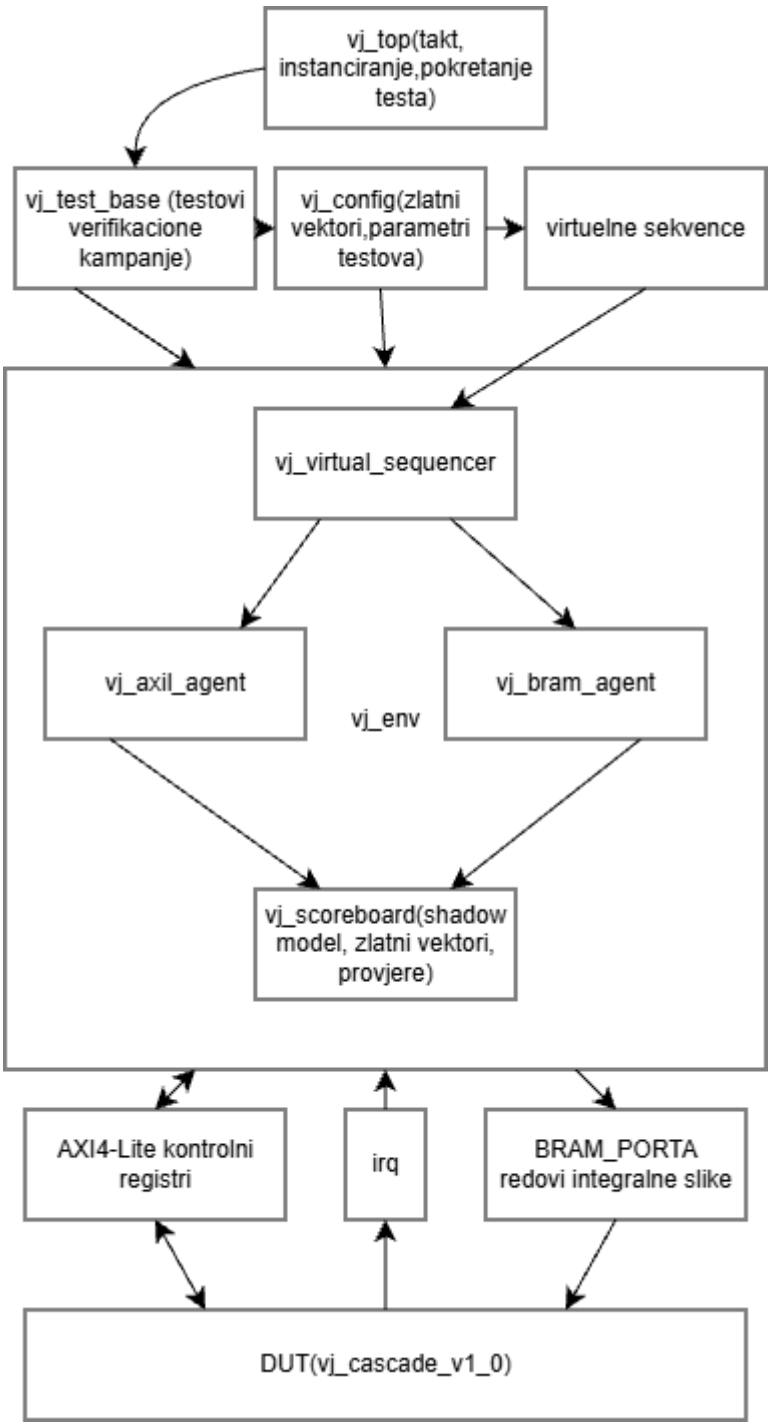
Stavka provjere	Mehanizam provjere	Test
RESULT odgovara referenci za svaki prozor	Poređenje sa zlatnim vektorima	vj_test_mode_a, vj_test_full
Prihvatanje prozora koji sadrži lice	Bin cp_result::accept	vj_test_mode_a
Odbijanje u ranim fazama kaskade	Bins reject_st0, reject_early	vj_test_full
Odbijanje u srednjim i kasnim fazama	Bins reject_mid, reject_late	vj_test_full
HIT_COUNT odgovara broju detekcija	Poređenje sa zlatnim vektorima	vj_test_mode_b
HIT_DATA vraća tačnu x koordinatu	Poređenje sa listom očekivanih detekcija	vj_test_mode_b
Sweep i pojedinačni prozori daju isti odgovor	Iste reference za oba režima	vj_test_full
Rotacija banaka ne mijenja rezultat	Invarijanta nad istim zlatnim vektorima	vj_test_rotation
Hit bafer se briše na početku svakog sweep-a	Dva identična sweep-a daju isti HIT_COUNT	vj_test_sweep_edge
Degenerisani sweep-ovi se uredno završavaju	HIT_COUNT i STATUS.BUSY	vj_test_sweep_edge

Tabela 8. Verifikacioni plan - funkcionalnost kaskade

3. Struktura verifikacionog okruženja

Za funkcionalnu verifikaciju korišćena je UVM metodologija, standardizovana od strane Accellera organizacije i široko prihvaćena u industriji. Osnovna ideja je da okruženje bude modularno, sa jasno definisanim ulogama komponenti, što omogućava ponovnu upotrebu i skalabilnost.

Okruženje sadrži dva agenta, virtuelni sekvencer i scoreboard. Podjela na dva agenta direktno prati podelu interfejsa dizajna.



Slika 1. Blok šema verifikacionog okruženja

3.1 Uloga virtuelnog sekvencera

Aktivnosti dva interfejsa se ne smiju preklapati. Redovi integralne slike uploaduju se kroz BRAM_PORTA isključivo dok je jezgro u idle stanju, a tek zatim se preko AXI4-Lite interfejsa pokreće obrada. Virtuelni sekvencer drži pokazivače na oba sekvencera, pa jedna virtuelna sekvenca vodi cijeli scenario redom, a ne paralelno:

reset -> upload redova (BRAM) -> konfiguracija registara (AXI) -> START (AXI) -> čekanje DONE -> čitanje rezultata (AXI).

3.2 Komponente okruženja

Komponenta	Datoteka	Uloga
vj_defs_pkg	config/vj_defs_pkg.sv	Registarska mapa i konstante dizajna
vj_config	config/vj_config.sv	Konfiguracija i učitavanje zlatnih vektora
vj_image_case	config/vj_image_case.sv	Jedan test slučaj sa zlatnim vektorima
vj_if	vj_if.sv	Virtuelni interfejs, oba interfejsa DUT-a i irq
vj_axil_seq_item	agent_axil/	Upis, čitanje, reset ili pauza sa timing poljima
vj_axil_driver	agent_axil/	Jedina komponenta koja mijenja signale magistrale
vj_axil_monitor	agent_axil/	Rekonstrukcija transakcija i pet coverage grupa
vj_bram_seq_item	agent_bram/	Jedan upis riječi (banka, kolona, podatak)
vj_bram_driver	agent_bram/	Kontroliše BRAM_PORTA kao AXI BRAM kontroler
vj_bram_monitor	agent_bram/	Dekodovanje banke i kolone iz bajt adrese
vj_virtual_sequencer	seq/	Pokazivači na oba sekvencera
vj_scoreboard	vj_scoreboard.sv	Sve provjere, shadow model i zlatni vektori
vj_env	vj_env.sv	Instanciranje i povezivanje komponenti
vj_tests	tests/vj_tests.sv	Devet testova regularne kampanje

Tabela 9. Komponente verifikacionog okruženja

3.3 Redoslijed kompajliranja paketa

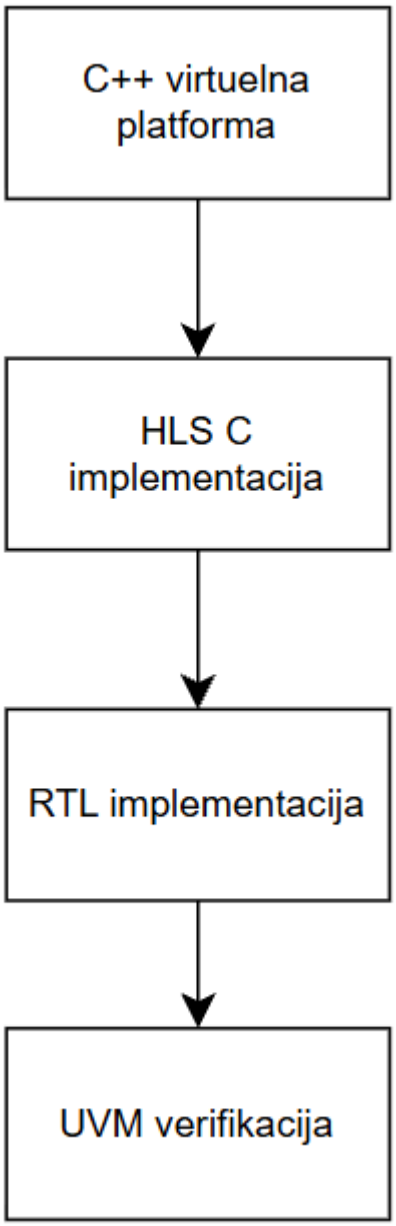
Paketi se međusobno uvoze određenim redoslijedom koji je bitan pri kompajliranju. Redoslijed je isti u obje simulatorske skripte:

vj_defs_pkg
-> vj_config_pkg
-> vj_axil_agent_pkg
-> vj_bram_agent_pkg
-> vj_seq_pkg
-> vj_test_pkg
-> vj_if.sv, vj_top.sv

4. Referentni model i zlatni vektori

Pravilo u verifikaciji hardvera glasi da se izlaz dizajna koji se verifikuje ne smije koristiti kao referenca za provjeru tog istog dizajna. Ako dizajn ima grešku, ta greška bi bila i u referenci, pa bi sve izgledalo ispravno.

Referenca u ovom projektu su zlatni vektori generisani softverskim modelom sa virtuelne platforme razvijene na predmetu Projektovanje elektronskih uređaja na sistemskom nivou. Lanac razvoja je sljedeći:



Slika 2. Lanac razvoja i porijekla referentnih vrijednosti

Referenca dakle potiče dva nivoa apstrakcije iznad RTL opisa koji se verifikuje, sa zasebnog predmeta. Time je zahtjev za nezavisnošću reference zadovoljen u jačem obliku nego što bi bio da je model razvijan potpuno uporedo sa hardverom.

4.1 Skupovi zlatnih vektora

Korišćena su dva skupa vektora. Prvi je kreiran uz RTL opis i koristi se za brzu regresiju. Drugi je izveden iz test vektora HLS implementacije i koristi se za mjerenje pokrivenosti.

Ključno zapažanje koje je omogućilo konverziju jeste da jedan snimak iz HLS testbencha sadrži tačno 25 redova integralne slike i 25 redova kvadrirane integralne slike za jednu y poziciju, a RTL banke drže baš tih 25 integralnih redova i dva kvadrirana. Snimak se zato preslikava direktno u sadržaj banaka, bez ikakvog preračunavanja.

Svojstvo	Originalni skup	HLS skup	Odnos
Broj slučajeva	5	5	-
Broj prozora	91	833	9,2×
Prihvaćenih prozora	2	26	13×
Najdublje odbijanje	faza 4	faza 24	-
Broj skala	4	4	-
Vrijeme simulacije (vj_test_full)	769 µs	7,41 ms	9,6×

Tabela 10. Poređenje skupova zlatnih vektora

4.2 Shadow model registarskog fajla

Pored zlatnih vektora, scoreboard održava i softversku kopiju svakog registra sa mogućnošću upisa, uz odsijecanje na stvarnu širinu polja u jezgru. To je drugi, nezavisni referentni model, izveden iz specifikacije registarske mape, a ne iz dizajna.

Shadow model omogućava provjeru koju zlatni vektori ne mogu pokriti: da je vrijednost upisana u registar zaista sačuvana, odsječena na pravu širinu, i da upis u read-only registre nema efekta.

5. Testovi

Implementirano je devet osnovnih testova za regularnu verifikacionu kampanju, poređanih od najjednostavnijeg prema najsloženijem. Svaki naredni test pretpostavlja da je prethodni prošao.

Svi testovi naslijeđuju baznu klasu vj_test_base koja kreira konfiguraciju i okruženje. Konkretni testovi samo biraju virtuelnu sekvencu, pa dodavanje novog testa zahtijeva klasu od desetak linija koda bez ijedne izmjene u ostatku okruženja.

Test	Šta verifikuje
vj_test_reset	Reset vrijednosti svih registara
vj_test_regs	Registarski fajl, širine polja, varijacije handshake-a
vj_test_ctrl	START kao impuls, DONE sticky, irq, soft reset
vj_test_mode_a	Kaskada u režimu A, poređenje po prozoru
vj_test_mode_b	Sweep i hit bafer
vj_test_sweep_edge	Degenerisani i uzastopni sweep-ovi
vj_test_rotation	Rotacija banaka za ROW_BASE 0, 1, 12 i 24
vj_test_full	Svi slučajevi, oba režima — glavni test
vj_test_random	Nasumični stimulus za punjenje pokrivenosti

Tabela 11. Pregled testova regularne verifikacione kampanje

5.1 Test rotacije kao invarijanta

Test vj_test_rotation zaslužuje poseban osvrt jer koristi invarijantu umjesto nove reference. Ako softver logički red r upiše u fizičku banku (r + R) mod 25 i u registar ROW_BASE upiše R, jezgro mora dati identične rezultate kao za R jednako nuli.

Time se mehanizam rotacije verifikuje bez ijednog novog zlatnog vektora, jer ista referenca važi za svaku rotaciju. Test je izvršen za četiri vrijednosti ROW_BASE i sve 142 pozicije prozora, ukupno 568 poređenja.

6. Rezultati verifikacije

Regresija obuhvata trinaest pokretanja - osam direktnih testova i pet pokretanja nasumičnog testa sa različitim seedovima. Sva pokretanja su prošla bez ijedne UVM greške ili fatalne poruke.

Ukupno je izvršeno 1.405.924 provjera, bez ijednog neslaganja sa referentnim modelima.

Test	Provjera	Referenca	Hit	Presk.	Status
vj_test_reset	119	-	-	0	PASS
vj_test_regs	109	-	-	0	PASS
vj_test_ctrl	247	-	-	0	PASS
vj_test_mode_a	71.706	142	-	0	PASS
vj_test_mode_b	69.573	-	7	0	PASS
vj_test_sweep_edge	130.288	-	16	0	PASS
vj_test_rotation	286.824	568	-	0	PASS
vj_test_full	582.230	833	31	0	PASS
vj_test_random (5 seedova)	264.828	13	23	12	PASS
UKUPNO	1.405.924	1.556	77	12	PASS

Tabela 12. Rezultati regresije

Kolona „Presk.” broji čitanja algoritamskih registara za koja ne postoji zlatna referenca. Takva čitanja se javljaju isključivo u nasumičnom testu, tokom faze nestrukturiranog registarskog saobraćaja prije nego što je pokrenuta ijedna obrada. Scoreboard ih ne poredi, ali ih broji, jer bi tiho preskočena provjera inače prošla nezapaženo.

Scoreboard na kraju svake simulacije ispisuje izvještaj sledećeg oblika:

```
UVM_INFO D:/UVMProjectfinal/vj_uvm/verif/vj_scoreboard.sv(584) @ 942005000: uvm_test_top.env.sb [VJ_SB] ================================================
UVM_INFO D:/UVMProjectfinal/vj_uvm/verif/vj_scoreboard.sv(585) @ 942005000: uvm_test_top.env.sb [VJ_SB]         VIOLA-JONES SCOREBOARD REPORT
UVM_INFO D:/UVMProjectfinal/vj_uvm/verif/vj_scoreboard.sv(586) @ 942005000: uvm_test_top.env.sb [VJ_SB] ================================================
UVM_INFO D:/UVMProjectfinal/vj_uvm/verif/vj_scoreboard.sv(587) @ 942005000: uvm_test_top.env.sb [VJ_SB] Ukupno provjera            : 82304
UVM_INFO D:/UVMProjectfinal/vj_uvm/verif/vj_scoreboard.sv(588) @ 942005000: uvm_test_top.env.sb [VJ_SB]   registarskih read-backova   : 11228
UVM_INFO D:/UVMProjectfinal/vj_uvm/verif/vj_scoreboard.sv(589) @ 942005000: uvm_test_top.env.sb [VJ_SB]   RESULT vs zlatni vektor  : 4
UVM_INFO D:/UVMProjectfinal/vj_uvm/verif/vj_scoreboard.sv(590) @ 942005000: uvm_test_top.env.sb [VJ_SB]   hit buffer vs referenca  : 8
UVM_INFO D:/UVMProjectfinal/vj_uvm/verif/vj_scoreboard.sv(591) @ 942005000: uvm_test_top.env.sb [VJ_SB] Pokretanja jezgra, mod A   : 4
UVM_INFO D:/UVMProjectfinal/vj_uvm/verif/vj_scoreboard.sv(592) @ 942005000: uvm_test_top.env.sb [VJ_SB] Pokretanja jezgra, mod B   : 4
UVM_INFO D:/UVMProjectfinal/vj_uvm/verif/vj_scoreboard.sv(593) @ 942005000: uvm_test_top.env.sb [VJ_SB] Upisa u row banke          : 18619
UVM_INFO D:/UVMProjectfinal/vj_uvm/verif/vj_scoreboard.sv(594) @ 942005000: uvm_test_top.env.sb [VJ_SB] Citanja bez reference      : 5
UVM_INFO D:/UVMProjectfinal/vj_uvm/verif/vj_scoreboard.sv(595) @ 942005000: uvm_test_top.env.sb [VJ_SB] Neslaganja                : 0
UVM_INFO D:/UVMProjectfinal/vj_uvm/verif/vj_scoreboard.sv(596) @ 942005000: uvm_test_top.env.sb [VJ_SB] ------------------------------------------------
UVM_INFO D:/UVMProjectfinal/vj_uvm/verif/vj_scoreboard.sv(598) @ 942005000: uvm_test_top.env.sb [VJ_SB] REZULTAT: PASS
UVM_INFO D:/UVMProjectfinal/vj_uvm/verif/vj_scoreboard.sv(602) @ 942005000: uvm_test_top.env.sb [VJ_SB] ================================================
UVM_INFO D:/UVMProjectfinal/vj_uvm/verif/tests/vj_tests.sv(81) @ 942005000: uvm_test_top [vj_test_random] ================================================
```

Slika 3. Izvještaj scoreboard-a za test vj_test_random (dio regresije)

Nakon pokretanja regresije, moguće je izmjeriti i pokrivenost. Rezultati mjerenja pokrivenosti biće detaljnije analizirani u narednom poglavlju, a u nastavku je data slika iz HTML datoteke

koju program generiše za analizu pokrivenosti:

XILINX

Date	08/24/26 17:50:37 Central European Daylight Time
User	DT User
Version	Vivado Simulator Coverage Report 2022.2
Command Line	C:\Xilinx\Vivado\2022.2\bin\unwrapped\win64.o\xcrg.exe -report_format html -dir D:/UVMProjectfinal/vj_uvm/vivado_project/vj_uvm.sim/sim_1/behav/xsim/xsim.covdb -report_dir D:/UVMProjectfinal/vj_uvm/coverage
Number of Tests	1

Total Groups Coverage Summary

Score	Inst Score
99.5	99.5

Slika 4. Izvještaj o pokrivenosti - osnovni prozor

XILINX

Dashboard | Groups

Groups Coverage Summary

Score	Inst Score
99.5	99.5

Total groups in report: 8

Name	Score	Num Instances	Avg Instances Score	Weight	Goal	Merge Instances	Get Inst Coverage	Per Instance	Auto Bin Max	Comment
vj_axil_agent_pkg::vj_axil_monitor::cg_axil_write	100	1	100	1	100	0	0	1	64	
vj_axil_agent_pkg::vj_axil_monitor::cg_ctrl_write	100	1	100	1	100	0	0	1	64	
vj_axil_agent_pkg::vj_axil_monitor::cg_params_write	100	1	100	1	100	0	0	1	64	
vj_axil_agent_pkg::vj_axil_monitor::cg_axil_read	100	1	100	1	100	0	0	1	64	
vj_axil_agent_pkg::vj_axil_monitor::cg_result	96	1	96	1	100	0	0	1	64	
vj_bram_agent_pkg::vj_bram_monitor::cg_bram_write	100	1	100	1	100	0	0	1	64	
vj_test_pkg::vj_scoreboard::cg_checked_results	100	1	100	1	100	0	0	1	64	
vj_test_pkg::vj_scoreboard::cg_checked_hits	100	1	100	1	100	0	0	1	64	

Slika 5. Izvještaj o pokrivenosti - po covergroupovima

Sa slike se jasno vidi da je pokrivenost zanemarljivo narušena u grupi **cg_result** , konkretno u coverpointu **cp_hit_count** koji provjerava popunjenost hit buffera.

Group - vj_axil_agent_pkg::vj_axil_monitor::cg_result

Group Info : cg_result

Score	Num Insts	Avg Insts Score	Weight	Goal	Merge Insts	Get Inst Crvg	Per Inst	Auto Bin Max	Print Missing	Comment
96	1	96	1	100	0	0	1	64	0	

Source File(s) : D:/UVMProjectfinal/vj_uvm/verif/agent_axil/vj_axil_monitor.sv

Group Instance[s] Info : 1 (Total)

Name	Score	Weight	Goal	At Least	Auto Bin Max	Print Missing	Comment
cg_resu ...	96	1	100	1	64	0	

Instance : cg_result_13

Instance Info : cg_result_13

Score	Weight	Goal	At Least	Auto Bin Max	Print Missing	Is Instantized	Comment
96	1	100	1	64	0	1	

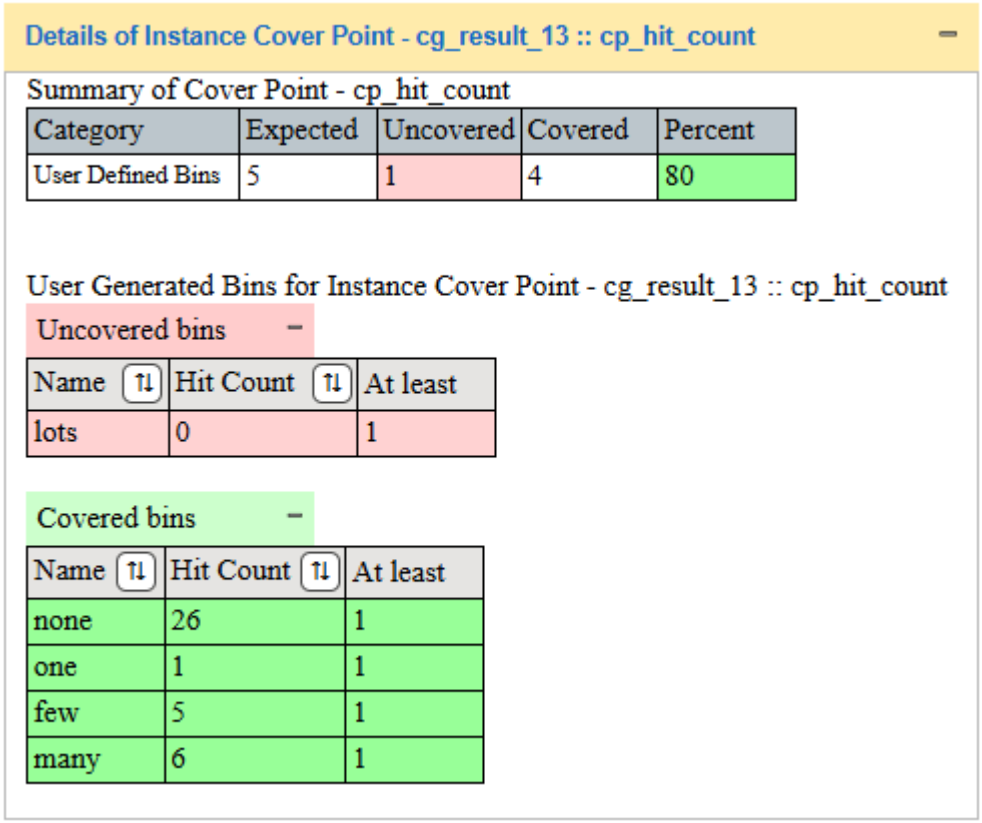
Instance Variable[s] Summary : cg_result_13

Cover Points	Category	Expected	Uncovered	Covered	Avg Percent
5	Variables	16	1	15	96

Instance Cover Point[s] Details : cg_result_13

Name	Expected	Uncovered	Covered	Percent	Goal	Weight	At Least	Auto Bin Max	Comment
cp_resu ...	5	0	5	100	100	1	1	64	
cp_hit_ ...	5	1	4	80	100	1	1	64	
cp_busy	2	0	2	100	100	1	1	64	
cp_done	2	0	2	100	100	1	1	64	
cp_mode ...	2	0	2	100	100	1	1	64	

Slika 6. Izvještaj o pokrivenosti - analiza nepokrivene covergroupe



Details of Instance Cover Point - cg_result_13 :: cp_hit_count

Summary of Cover Point - cp_hit_count

Category	Expected	Uncovered	Covered	Percent
User Defined Bins	5	1	4	80

User Generated Bins for Instance Cover Point - cg_result_13 :: cp_hit_count

Uncovered bins

Name	Hit Count	At least
lots	0	1

Covered bins

Name	Hit Count	At least
none	26	1
one	1	1
few	5	1
many	6	1

Slika 7. Izvještaj o pokrivenosti - analiza coverpointa

Nepokriven je bin **lots** , koji se strukturno **ne može pokriti** jer se odnosi na hit buffer, koji je strukturno pravljen da bude veličine 512 mjesta, dok on ima teorijski maksimalno 336 unosa (zavisno od broja redova koji imaju detekciju), pa je on nedostižan i nije problematičan.

7. Analiza funkcionalne pokrivenosti

Pokrivenost je mjerena tri puta, nakon svake značajne izmjene okruženja ili skupa vektora. Razlika između tih mjerenja je sadržajniji rezultat od bilo kog pojedinačnog procenta.

Mjerenje	Iteracija 1	Iteracija 2	Iteracija 3
Testni vektori	originalni	HLS	HLS
Ispravke okruženja	nema	A1, A2, A3	A1–A3 + marker
Ukupna pokrivenost	87,29 %	98,92 %	99,50 %
Grupa na 100 %	4 / 8	6 / 8	7 / 8

Tabela 13. Pokrivenost kroz tri iteracije mjerenja

7.1 Pokrivenost po grupama

Covergroup	Lokacija	Iter. 1	Iter. 3
cg_axil_write	monitor AXI4-Lite	88,33 %	100 %
cg_ctrl_write	monitor AXI4-Lite	100 %	100 %
cg_params_write	monitor AXI4-Lite	100 %	100 %
cg_axil_read	monitor AXI4-Lite	100 %	100 %

cg_result	monitor AXI4-Lite	70 %	96 %
cg_bram_write	monitor BRAM	100 %	100 %
cg_checked_results	scoreboard	90 %	100 %
cg_checked_hits	scoreboard	50 %	100 %

Tabela 14. Pokrivenost po covergroup-ama

7.2 Nedostaci okruženja otkriveni mjerenjem pokrivenosti

Tri prethodno nepokrivena bina bila su posljedica nedostatka u testbenchu, a ne u dizajnu. Nijedan test ih nije prijavio - svih 152.000 provjera prve iteracije je prolazilo.

Oznaka	Bin	Uzrok	Ishod
A1	cp_busy::busy	STATUS se čitao samo posle DONE, pa BUSY nikada nije viđen na jedinici	1.579 pogodaka
A2	ctrl × addr_first / data_first	Svi upisi u CTRL sa podrazumijevanim timingom	pokriveno
A3	cp_bready::immediate	Driver je BREADY podizao prekasno, bin nedostižan po konstrukciji	pokriveno

Tabela 15. Ispravke okruženja izvedene iz analize pokrivenosti

Slučaj A3 zaslužuje detaljan osvrt. Jedna od prethodnih ispravki bila je pogrešna i izazivala je timeout simulacije. Podizanje signala BREADY prije nego što slave prihvati upis navelo je slave da signal BVALID obori već u sljedećem taktu:

```
IF wr_en = '1' THEN bvalid_r <= '1';
ELSIF S_AXI_BREADY = '1' AND bvalid_r = '1' THEN bvalid_r <= '0';
```

Master, koji odgovor traži takt kasnije, tako ga nikada nije vidio. Simulacija je stajala do isteka vremenskog ograničenja od 500 ms, bez ijedne poruke o uzroku. Konačno rješenje podiže BREADY tačno na ivici prihvatanja upisa.

Posljedica te situacije je sljedeća sigurnosna trajna izmjena okruženja - sva čekanja na handshake u drajveru dobila su gornju granicu od 1000 taktova, uz poruku koja imenuje koji handshake nije završen i na kom registru. Testbench koji se zaglavi umesto da prijavi grešku predstavlja defekt testbencha.

7.3 Nazadovanje pokrivenosti izazvano boljim vektorima

U drugoj iteraciji cross x_bank_col pao je sa 135 na 110 pokrivenih binova, iako su coverpoint-i cp_bank i cp_col pojedinačno ostali na sto posto.

Uzrok je bio u sadržaju vektora. Originalni skup je sadržao skalu širine 360, pa je kolona 359 redovno upisivana. Izabrani HLS snimci imaju najveću širinu 245, pa tu kolonu dosežu samo nasumični upisi, svega dva puta, što ostavlja 25 kombinacija banka × kolona praznim.

Zamjena vektora je dakle zatvorila dva bina i otvorila dvadeset pet drugih. Zaključak je da pokrivenost nije monotona u odnosu na kvalitet stimulusa: poboljšanje u jednoj dimenziji

može oslabiti drugu, a jedini način da se to primjeti jeste ponovno mjerenje posle svake izmjene.

Ispravka je izvedena proširenjem sekvence vj_bram_marker_seq, koja sada upisuje marker u prvu, srednju i posljednju kolonu svake od 27 banaka. Ta sekvenca ne nosi sliku nego dokazuje da je adresna putanja dostupna u cijelom opsegu, nezavisno od širine izabranih skala.

7.4 Pokrivenost naspram stvarne provjere

Najinstruktivniji nalaz kampanje nije nijedan pojedinačni bin, nego odnos između pokrivenosti i stvarne provjere.

Pre nego što je dodat test vj_test_rotation, coverpoint cp_row_base bio je potpuno pokriven, jer sekvenca vj_axil_reg_seq upisuje vrijednosti 0, 7, 24 i 33. Uprkos punoj pokrivenosti tog coverpoint-a, mehanizam rotacije nije bio proveren nijednom provjerom: nijedan test nije pokretao jezgro sa vrijednošću ROW_BASE različitom od nule. Registar je bio pisan i čitan, ali njegov efekat na ponašanje dizajna nije mjeren.

Upravo zbog toga scoreboard sadrži zasebne covergroup-e cg_checked_results i cg_checked_hits, koje sampluju isključivo transakcije za koje postoji zlatna referenca. Razlika između tih grupa i grupa u monitoru predstavlja direktnu mjeru koliko je od izvršenog zaista i provjereno. Cross x_case_mode sa deset od deset pokrivenih binova je jedini podatak u izvještaju koji tvrdi da je svaka testna slika provjerena u oba režima rada u odnosu na nezavisni referentni model.

8. Regresioni testovi

Regresija je automatizovana Tcl skriptom koja redom pokreće sve testove, svaki u svoju bazu pokrivenosti, i na kraju ispisuje zbirni pregled. Nasumični test se pušta sa pet različitih seedova, jer je to jedini način da nasumični stimulus zaista popuni binove koje direktni testovi ne dosežu.

Skripta obavezno poziva close_sim prije svakog pokretanja. Bez toga simulator ponovo koristi keširani snapshot sa starim imenom testa, pa se izvršava prethodni test, što bi mogao biti uzrok zbunjujućih rezultata.

Koraci za regresione testove u Vivadu:

```
source run.tcl          ;# kreiranje projekta i prvo pokretanje


source regression.tcl   ;# pokretanje 13 testova
source coverage.tcl     ;# objedinjeni HTML izvještaj
```

Koraci za regresione testove u Xcelimu:

```
./regression_xcelium.sh //za regresiju
```

9. Pokretanje simulacije

Okruženje se pokreće u dva simulatora bez ijedne izmjene izvornog koda. Putanja do datoteka sa podacima proslijeđuje se plusargom, umjesto da postoje zasebne verzije izvornih datoteka po simulatoru.

Prenosivost je potvrđena mjerenjem: svih 25 SystemVerilog datoteka kompajlirano je pod alatom Xcelium 19.03 iz 2019. godine bez ijedne greške, a kompletna regresija od trinaest testova prošla je bez ijednog neuspješnog testa. Test vj_test_full završava se u oba simulatora u identičnom simulacionom trenutku od 7.414.475 ns, što potvrđuje da okruženje ne sadrži race condition niti se oslanja na event scheduling specifično za jedan alat.

Za VHDL dio prevođenja pod Xcelium alatom neophodan je switch -v93, jer parser podrazumijevano koristi VHDL-87, dok RTL opis koristi konstrukcije uvedene standardom iz 1993. godine.

9.1 Vivado simulator (xsim)

```
cd <putanja>/vj_uvm/scripts

set ::vj_test vj_test_full
```

```
source run.tcl          ;# kreiranje projekta i prvo pokretanje

#Bitna stvar za Vivado : nakon prvog pokretanja moguce je koristiti i
alternativnu skriptu koja ne kreira Vivado projekat, sto stedi vrijeme.
Promjena testa bez ponovnog kreiranja projekta:

set ::vj_test vj_test_mode_b

source rerun.tcl ;# ponovno pokretanje bez kreiranja projekta
```

9.2 Xcelium simulator

```
cd <putanja>/vj_uvm/scripts

./run_xcelium.sh //za pokretanje pojedinacnog testa
Bitna stvar za Xcelium : drugi argument je naziv testa.

./regression_xcelium.sh //za regresiju
imc -load ../coverage_xcelium //za ucitavanje u IMC alatu (ako licenca
postoji)
```

Razlike između dva okruženja svedene su na skripte.

Stavka	Vivado	Xcelium
tb_vectors.txt	kopira se u radni direktorijum	plusarg +VJ_DATA_DIR
classifier_rom.mem	kopira run.tcl	kopira run_xcelium.sh
Izbor testa	set ::vj_test, pa rerun.tcl	argument skripte
Izvještaj pokrivenosti	xcrg, HTML	baze se prikupljaju; IMC nije instaliran

Tabela 16. Razlike u pokretanju između dva simulatora

10. Analiza grešaka

Verifikaciona kampanja nije otkrila nijednu grešku u dizajnu. Svih 1.405.924 provjera prošlo je bez neslaganja, na svim legalnim ulazima, u oba režima rada i na četiri skale.

To nije neočekivano. RTL opis je preuzet kao završen i prethodno testiran projekat sa predmeta Projektovanje složenih digitalnih sistema, uz postojeće VHDL testbencheve koji su pokrivali osnovni tok izvršavanja. Pozitivan rezultat rezultat na ispravnom dizajnu je očekivan ishod.

Analiza grešaka se zato može oslanjati na tri izvora: greške pronađene u samom verifikacionom okruženju, hipoteze o robusnosti dizajna izvedene statičkom analizom, i eventualno planirano injektiranje grešaka radi mjerenja sposobnosti okruženja da ih otkrije. Stavke 2 i 3 neće biti detaljnije razmatrana u ovom dokumentu, ali se mogu analizirati u okviru budućeg diplomskog rada na ovu temu.

10.1 Greške pronađene u verifikacionom okruženju

Tri greške u okruženju pronađene su tokom verifikacione kampanje. Sve tri su dokumentovane jer ilustruju mehanizme kojima se greške otkrivaju, i jer bi svaka od njih tiho oslabila verifikaciju da nije primjećena.

Greška	Kako je otkrivena	Posljedica da nije otkrivena
Kasno podizanje BREADY signala u driveru	Analiza pokrivenosti; bin cp_bready::immediate imao je nula pogodaka posle 152.000 uspešnih provjera	Scenario u kojem je master spreman prije slave-a nikada ne bi bio izvršen
Zastoj usled preranog podizanja BREADY signala	Simulacija je stala do isteka vremenskog ograničenja, bez poruke o uzroku	Testbench koji se zaglavljuje umesto da prijavi grešku
Pogrešno izračunata gornja granica sweep-a u konvertoru vektora	Poređenje sa dokumentacijom HLS testbencha	Provjera hit bafera bila bi tiho preskočena na svakoj y poziciji

Tabela 17. Greške pronađene u verifikacionom okruženju

10.2 Hipoteze iz statičke analize

Čitanjem RTL opisa formulisane su četiri hipoteze o nezaštićenim stanjima dizajna. Sve četiri se odnose na robusnost, a ne na funkcionalnost: za njihovo aktiviranje potrebno je da softver upiše vrijednost izvan dokumentovanog opsega.

Za svaku hipotezu razvijen je usmjereni test. Testovi u trenutku pisanja ovog dokumenta još nisu pokrenuti, pa se rezultati ne navode, te će oni biti predmet u diplomskom radu. Statička analiza pokazuje da nešto može poći po zlu, potvrdu ili opovrgavanje daje isključivo simulacija.

Oznaka	Hipoteza	Uslov aktiviranja
H1	rot_bank ne provjerava opseg; ROW_BASE 25–26 daje tiho pogrešnu banku, 27–31 pristup izvan opsega niza	ROW_BASE izvan opsega 0–24
H2	Uslov nastavka sweep-a je trajno tačan za nulti korak, pa se obrada ne završava	X_STEP = 0
H3	Sabiranje x_reg + x_step je devetobitno i omotava se po modulu 512	X_LAST blizu 511
H4	START upisan u toku obrade se tiho odbacuje	START dok je BUSY visok

Tabela 18. Hipoteze o robusnosti dizajna

10.3 Planirano injektiranje grešaka

Treći način moguće je takođe razmotriti ako dizajn ne sadrži greške koje bi kampanja mogla otkriti, tada se kvalitet verifikacionog okruženja mjeri injekcijom grešaka. Postupak bi bio da se u ispravan RTL opis unese tačno jedna izmjena, pokrene se cijela regresija, i bilježi se da li je bar jedan test pao, koji i sa kojom porukom.

Jedna potencijalno zanimljiva varijanta bi mogla biti uklanjanje zaštite od overflowa hit bafera. Za njeno aktiviranje bilo bi potrebno više od 512 detekcija u jednom sweep-u, a najšira skala ima svega 336 prozora, pa bi u nekoj široj implementaciji bilo interesantno razmotriti i to kako bi se sistem ponašao u ovoj situaciji.

11. Ograničenja verifikacije

Sljedeće stavke ostaju neprovjerene i predstavljaju granicu ove verifikacione kampanje. Ova lista je sastavni dio rezultata: precizno poznavanje granica verifikacije vrijednije je od tvrdnje o potpunosti.

Hit bafer sa 33 i više detekcija, kao i zaštita od njegovog overflowa. Najšira skala ima 336 prozora po sweep-u, pa se bafer dubine 512 ne može popuniti nijednim mogućim testom nad ovim ulaznim podacima.

Inkrementalni streaming. Svi izvršeni testovi prepisuju svih 27 banaka iz početka, dok realan sistem mijenja samo jedan red od 25 i pomjera ROW_BASE za jedan. Test vj_test_frame

pokriva taj scenario, ali u trenutku pisanja još nije pokrenut, ali nije ni neophodan kako bi se provjerila funkcionalnost sistema.

Prelazi između skala piramide. Svaka skala se testira zasebno; promjena registra WIDTH uz ponovno punjenje banaka nije scenario nijednog testa, dijelom i zbog toga što se registar WIDTH nakon RTL dizajn promjena više ne koristi.

Bajt enable putanja BRAM porta. Dizajn ignoriše pojedinačne bajt enable signale, a provjera nije moguća jer je izlaz bram_rddata_a vezan na nule.

Banke sa indeksima 27 do 31. Nalaze se u adresnom opsegu AXI BRAM kontrolera, ali fizički ne postoje, pa se upisi u njih tiho odbacuju.

12. Zaključak

Funkcionalna verifikacija hardverskog akceleratora detekcije lica upotrebom Viola-Jones metodologije sprovedena je primjenom UVM metodologije. Razvijeno je okruženje sa dva agenta, virtuelnim sekvencerom i scoreboard-om koji istovremeno koristi dva nezavisna referentna modela: shadow model registarskog fajla i zlatne vektore softverskog modela.

Regresijom od trinaest pokretanja izvršeno je 1.405.924 provjera bez ijednog neslaganja. Funkcionalna pokrivenost dostigla je 99,5 posto, pri čemu jedini nepokriveni bin nije dostižan nijednim mogućim testom nad postojećim ulaznim podacima.

Verifikacijom je potvrđeno sljedeće: ispravne reset vrijednosti svih registara i pravilno odsijecanje na širinu polja u jezgru; ispravan AXI4-Lite protokol uključujući sve varijante redoslijeda kanala i zadržavanje odgovora; ispravna semantika kontrolnih bitova, gdje je START impuls a DONE se briše isključivo upisom jedinice; potpuna dostupnost adresne putanje svih 27 banaka; poklapanje izlaza kaskade sa nezavisnim referentnim modelom za svih 833 prozora u oba režima rada; i invarijantnost rezultata na rotaciju row banaka.

Tokom kampanje otkrivene su tri greške u samom verifikacionom okruženju, od kojih je jedna bila vidljiva isključivo kroz analizu pokrivenosti, a ne kroz nijednu od 152.000 uspješnih provjera, što dokazuje da je visoka pokrivenost neophodan, ali ne i dovoljan uslov, a broj uspješnih provjera ne govori ništa o tome da li je neki scenario uopšte izvršen.

Literatura

Accellera Systems Initiative, „Universal Verification Methodology (UVM) 1.2 User Guide”, 2015.

P. Viola, M. Jones, „Rapid Object Detection using a Boosted Cascade of Simple Features”, IEEE CVPR, 2001.

C. Spear, „SystemVerilog for Verification: A Guide to Learning the Testbench Language”, Springer, 2012.

B. Wile, J. C. Goss, W. Roesner, „Comprehensive Functional Verification: The Complete Industry Cycle”, Elsevier, 2005.

ARM, „AMBA AXI and ACE Protocol Specification”, 2021.

Xilinx Inc., „AXI BRAM Controller v4.1 — LogiCORE IP Product Guide”, 2019.

Digilent, „Zybo Z7 Reference Manual”

Materijali sa predavanja i vježbi predmeta Funkcionalna verifikacija hardvera, FTN Novi Sad